Phase 3 Report

Group 8 Report

CMPT 276 – Introduction to Software Engineering

Cao Mark	xmc2@sfu.ca
Lu Phung	hnl8@sfu.ca
Meng Jiyang	jma289@sfu.ca
Mopewou Manuel	mam65@sfu.ca
Nguyen Nguyen	nnn4@sfu.ca

March 24, 2026

Simon Fraser University

Contents

1. Findings

Based on TA feedback, several improvements were made to the system.

- Phung Ngoc enhanced the user interface by adding a top task bar, emergency buttons, an instruction panel, and an ending screen displaying game statistics. The visual design of game elements was also improved to make the game more attractive to players. In addition, a bug was identified where traps and regular rewards could appear on the same cell; this issue was fixed to ensure correct game behavior.

- Nhat Nguyen adjusted the chasing behavior of moving enemies to be slower and less aggressive, resulting in a more balanced gameplay experience. Several visual elements were also redesigned to improve overall aesthetics. Furthermore, a level selection screen was introduced before starting the game, along with a structured level system and new maps.

- Mark Cao identified a design issue in the game loop during testing. Specifically, the `checkWin()` method was originally called after enemy movement, which was unnecessary since a win condition cannot occur at that stage. This redundant check was removed to simplify the turn flow and improve code clarity.

2. Unit Testing

Based on the individual tasks in Phase 2, each team member was responsible for writing unit tests for the classes they implemented. This ensured that each component was independently verified according to its intended functionality.

The tests focus heavily on the correctness and robustness of core data structures such as the board, cells, and positions. These tests verify that the board is initialized correctly from a given map, that all cell types (such as walls, barriers, start, and exit) are properly assigned, and that invalid positions are handled safely. Additionally, the system ensures that occupancy and item management behave correctly when actors move across the board.

Movement-related components are also thoroughly tested. This includes validating that actors and the main character can only move within valid boundaries, do not enter blocked cells, and correctly update their positions after each action. Special attention is given to maintaining the movement history of the player, as it is later used by moving enemies. Enemy behavior is tested to ensure that they follow the player's recorded path, respect movement constraints, and interact correctly with the player through encounter detection.

Item-related testing ensures that rewards and punishments behave as expected. Regular and bonus rewards must contribute the correct score and cannot be collected multiple times, while traps apply penalties only once and are properly removed after activation.

For the testing of GameState, ScoreCount, Result, and the Level-related classes, the tests cover their core functionalities, including basic operations, state updates, score calculations, and the generation of game results.

At a higher level, system-level components such as the game engine and input handler are tested to ensure correct initialization, proper turn sequencing, and accurate detection of win and loss conditions. The tests verify that the game progresses correctly across ticks, processes player input in the right order, and transitions between different game states appropriately.

3. Integration Testing

One important interaction tested is between the game engine and the game state, ensuring that each game tick correctly updates the player, enemies, and items in the proper order.

Another key interaction is between the level system and the game state, where levels are created and passed into the runtime system. The `LevelAndGameStateTest` verifies that levels are correctly initialized and integrated.

Additionally, interactions between the main character and items, as well as between the main character and enemies, are validated through integration-style scenarios to ensure that gameplay mechanics behave consistently.

4. Test Automation

All tests were implemented using JUnit and organized according to Maven conventions. Test classes are located in `src/test/java`, and test resources are stored in `src/test/resources`.

A total of 218 test cases were executed successfully with no failures or errors, demonstrating that the system behaves as expected under the tested conditions.

5. Test Quality and Coverage

Core components such as the engine, actor system, and model layer achieve high coverage, indicating that most critical logic paths are well tested.

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
ui		0%		0%	103	103	675	675	37	37	6	6
model		90%		84%	17	93	15	167	0	35	0	4
trap		92%		67%	41	88	20	146	0	24	0	3
engine		95%		80%	24	123	8	214	1	64	1	9
actor		94%		81%	22	89	0	156	0	27	0	4
Item		94%		69%	23	52	13	94	0	14	0	5
default		0%		n/a	3	3	5	5	3	3	1	1
LevelSection		97%		87%	1	22	1	42	0	17	0	5
Total	4,120 of 7,275	43%	250 of 687	63%	234	573	737	1,499	41	221	8	37

Figure 1: Test Coverage Report

However, some parts of the system remain partially uncovered. In particular, certain branches related to null checks and defensive programming are intentionally not covered.

These branches are designed to handle invalid states that cannot occur under normal gameplay due to constraints enforced by the system.

For example, the game cannot start without selecting a valid level, which makes certain null-related branches in classes such as `LevelFactory` unreachable during normal execution, as identified by Ji. As a result, the only uncovered part is the default branch in LevelFactory.

Moreover, some branches in the Board class, particularly within loadFromMap, parseMapStream, and charToTitle, are only executed under specific invalid input conditions (e.g., missing start/exit cells, inconsistent map formats, or unknown characters), which require dedicated test files to trigger.